

Deep Learning for RSSI-Based Passenger Movement Classification in Public Transport

Guilherme Matos*, André Ribeiro*, Julio Corona*, Mário Antunes*[†] and Diogo Gomes*[†]

*Instituto de Telecomunicações, Universidade de Aveiro, Aveiro, Portugal

E-mails: {gui.matos, andrepedroribeiro, jcamejo, mario.antunes, dgomes}@av.it.pt

[†]DETI, Universidade de Aveiro, Aveiro, Portugal

E-mails: {mario.antunes, dgomes}@ua.pt

Abstract—Accurate, non-intrusive monitoring of passenger flows is essential for intelligent public transport systems. Prior work demonstrated that temporal sequences of Wi-Fi Received Signal Strength Indicator (RSSI) measurements can classify passenger movements using classical machine learning, achieving a Matthews Correlation Coefficient (MCC) of 0.756 on combined datasets. This paper extends that line of research by systematically evaluating deep learning architectures, including recurrent, convolutional, transformer, state-space, and mixture-of-experts models, for the same classification task across four dataset variants. We introduce 11 architectures, a class-conditioned data augmentation strategy expanding 2,253 samples to 10,115, and comprehensive hyperparameter optimization with 50 Optuna trials per model. Our best ensemble model, a two-stage deep stacking architecture, achieves MCC of 0.859 and accuracy of 89.4% on combined data, a 13.6% relative improvement over classical baselines. We further analyze the impact of multi-device interference, per-class confusion patterns, and the effectiveness of metadata-aware architectures in distinguishing true movement from environmental noise.

Index Terms—Passenger counting, RSSI fingerprinting, Wi-Fi sensing, public transport, deep learning, state space models, mixture of experts, hyperparameter optimization

I. INTRODUCTION

With over 75% of EU citizens living in urban areas and the transport sector accounting for nearly a quarter of greenhouse gas emissions [1], accurate passenger flow knowledge is fundamental to network optimization and dynamic scheduling.

Traditional Automatic Passenger Counting (APC) relies on cameras, infrared sensors, or manual counting, with tradeoffs in cost, privacy, and deployment complexity. Wi-Fi sensing offers a privacy-preserving alternative. By analyzing Received Signal Strength Indicator (RSSI) patterns from passengers' devices, operators can infer movement without active user participation or additional hardware beyond existing access points.

RSSI measurements are inherently noisy, affected by multipath propagation, device heterogeneity, and interference from concurrent transmissions. In prior work [2], we introduced a controlled dataset simulating four passenger movements (inside bus AA, boarding BA, alighting AB, at stop BB). Classical ML achieved Matthews Correlation Coefficient (MCC) 0.756 on combined data using Gaussian Processes. This paper investigates whether modern Deep Learning (DL) architectures

can better capture RSSI temporal dynamics, particularly under noisy multi-device conditions.

Our contributions are as follows. First, we systematically evaluate 11 DL architectures across four dataset variants with increasing interference complexity, including state-space models (Mamba-3), mixture-of-experts, and two-stage stacking ensembles. Second, we propose a class-conditioned data augmentation strategy that expands 2,253 to 10,115 samples through targeted Gaussian noise injection and intra-class Mixup. Third, we conduct Hyperparameter Optimization (HPO) via 50 Optuna trials per model spanning 19 search spaces. Fourth, we present a detailed interference analysis quantifying cross-route noise impact through heatmaps, trajectory visualization, and per-timestep perturbation measurements.

The remainder of this paper covers related work (Section 2), dataset and methodology (Section 3), experimental results (Section 4), and conclusions (Section 5).

II. RELATED WORK

Passenger counting spans multiple sensing modalities. Video-based APC using YOLOv5 achieves high accuracy but degrades under occlusion and variable lighting [3]. Edge AI cameras address scalability [4]. ML pipelines on traditional APC data enable offline correction of sensor errors [5].

Wi-Fi sensing avoids the cost and privacy drawbacks of cameras. iABACUS [6] demonstrated counting via Wi-Fi probe requests. Myrvoll et al. [7] used Wi-Fi signature counts for ridership estimation. Fabre et al. [8] applied random forests and gradient boosting to Wi-Fi sensor data. Simoncic et al. [9] monitored presence through passive probe requests. CSI-based methods [10] offer finer signal characterization but require specialized hardware unavailable on commodity access points. RSSI fingerprinting for localization has been extensively surveyed [11].

Recent advances in sequence modeling inform our architectural choices. Mamba [12] introduced selective state-space models with $O(L)$ complexity. Mamba-3 MIMO [13] extended this paradigm with multi-input multi-output capabilities. Autoformer [14] replaced self-attention with FFT-based auto-correlation mechanisms. Mixture of Experts [15] enables conditional computation through sparse activation.

Our prior work [2] established classical ML baselines on this dataset. Across 34 algorithms, Gaussian Processes achieved the best combined MCC of 0.756. This paper extends that analysis with 11 DL architectures, comprehensive HPO via Optuna [16], and systematic interference analysis.

III. MATERIALS AND METHODS

A. Dataset and Feature Engineering

The dataset simulates public transport in a controlled environment [2]. Zone A represents the vehicle interior with an access point near the door. Zone B represents the bus stop in an adjacent corridor. The same four devices used in the latest experiment [2] execute four movement patterns, denoted AA (inside vehicle), BB (at stop), BA (boarding), and AB (alighting). Each device produces 10 RSSI readings at 1 s intervals per observation.

Two scenarios were collected. The pure scenario uses a single device with no interference (160 samples). The noise scenario uses four concurrent devices executing different route combinations (1,200 samples). The combined dataset contains 2,253 labeled samples distributed as AA 578, AB 550, BA 546, and BB 577. The noise-only subset contains 1,196 samples (AA 298, AB 300, BA 298, BB 300).

Four engineered channels are extracted from the raw RSSI sequence r_t . The raw RSSI r_t provides absolute signal position. Velocity $\Delta_t = r_t - r_{t-1}$ captures movement speed. Acceleration Δ_t^2 captures rate of change. Window deviation $d_t = r_t - \mu_w$, where μ_w is the per-sample mean, captures trajectory shape independent of absolute RSSI level. These channels yield input tensors of shape $(N, 10, 4)$.

B. Proposed Architectures

Table I lists the 11 evaluated architectures. All models output 4-class logits through a linear classification head.

Table I
DL ARCHITECTURES EVALUATED

Architecture	Type	Key characteristic
RNN, GRU, LSTM, BiLSTM	Recurrent	2 layers, hidden 64
TCN [17]	Convolutional	Dilated causal, residual
Transformer	Attention	4-head self-attention, sinusoidal PE
MambaClassifier [13]	State-space	$O(L)$ complexity, rank- $R=16$
CNN2DRNN	Hybrid 2D+RNN	Conv2d (3, 3), (5, 3) $\rightarrow$ LSTM/GRU + attention
MetaFusion	Context-aware	LSTM/GRU fused with learned vector representations (dim 8)
Multi-Scale CNN	Multi-branch	Kernels (3, 5, 7) + residual + attention
SoftMoE	Mixture of Experts	Heterogeneous experts, learned top-2 gating routing
Autoformer [14]	Decomposition	FFT auto-correlation, trend-seasonal blocks
DeepStackEnsemble	Two-stage meta	18 bases $\rightarrow$ per-class Level2Net $\rightarrow$ residual MLP

The DeepStackEnsemble uses per-class Level2Nets: small MLPs receiving concatenated logits from the base models and outputting 4-class logits. Table II lists the 18 base models (2 configurations per architecture) that feed the ensemble.

C. Augmentation and Implementation

Two strategies expand the training data from 2,253 to 10,115 samples. Class-conditioned Gaussian noise adds 6 copies to clean samples ($\sigma = 3.0$ dBm) and 4 copies to noisy samples ($\sigma = 0.8$ dBm). Intra-class Mixup blends 600 random pairs per class as $X_{\text{new}} = \lambda X_1 + (1 - \lambda) X_2$ with $\lambda \sim \text{Beta}(0.4, 0.4)$, restricted to same-label pairs.

Table II
DEEPSTACK BASE MODELS

Architecture	Config A (light)	Config B (heavy)
RNN	dim=64, L=2, d=0.2	dim=256, L=4, d=0.3, bidir
GRU	dim=64, L=2, d=0.2	dim=256, L=3, d=0.3, bidir
LSTM	dim=64, L=2, d=0.2	dim=256, L=3, d=0.3, bidir
CNN	dim=64, blk=2, k=[3,5]	dim=256, blk=4, k=[3,5,7,9]
TCN	dim=128, d=3, k=4	dim=512, d=7, k=8
Transformer	dim=64, H=4, L=2	dim=256, H=8, L=4
BiLSTM	dim=64, L=2, d=0.2	dim=256, L=3, d=0.3
CNN2DRNN	Conv2d $\rightarrow$ GRU, d=0.2	Conv2d $\rightarrow$ LSTM, d=0.3
Mamba	d_model=64, L=2	d_model=256, L=4

Table III
DATASET VARIANTS

Variant	Train	Test	Condition
Pure	128	32	Single device, no interference
Noise	960	240	Four concurrent devices
Normal	1,802	451	Original unbalanced combined
Augmented	8,090	2,023	Synthetic samples added to training

The pipeline uses PyTorch 2.6 and Optuna 4.7. Training uses AdamW optimizer with cosine annealing, early stopping (patience 25), label smoothing ($\epsilon = 0.1$), Mixup augmentation ($\alpha = 0.2$), L1 regularization ($\lambda = 10^{-5}$), L2 weight decay (10^{-4}), dropout ($p = 0.3$), gradient clipping at 1.0, and batch size 64. HPO uses 50 Optuna TPE trials per model. SoftMoE additionally uses NSGA-II multi-objective optimization. The primary metric is MCC [18]. All experiments use 5-fold stratified cross-validation with 80/20 splits and 3 random seeds (3, 5, 42). A total of 65 models were evaluated.

IV. EXPERIMENTAL RESULTS

A. Experimental Setup

Table III describes the four dataset variants. Each stresses a different aspect of model robustness, from clean single-device recording to dense multi-device interference and synthetic augmentation. All models use 5-fold stratified cross-validation with 80/20 splits. Results are reported at seed 42 unless stated otherwise.

B. Variant-Level Comparison

Table IV lists the best model per variant. Pure yields RNN at 0.881 (seed 3); noise degrades to DS RNN_B at 0.798; Normal reaches DeepStackEnsemble at 0.859; augmentation lifts HPO TCN to 0.865.

Table IV
BEST MODEL PER VARIANT

Variant	Model	MCC	Acc.
Pure	RNN	0.881	0.906
Noise	DS RNN_B	0.798	0.846
Normal	DeepStackEnsemble	0.859	0.894
Augmented	HPO TCN	0.865	0.898

Table V
DL VS. CLASSICAL ML (MCC)

Variant	Classical best	DL best	Change
Combined	0.756 (Gaussian Process)	0.859 (DeepStack)	+13.6%
Pure	0.922 (KNN)	0.881 (RNN)	−4.4%
Noise	0.770 (CatBoost)	0.798 (DS RNN_B)	+3.6%

Table VI
TOP MODELS BY VARIANT

Model	Variant	MCC	Acc.
DeepStackEnsemble	Normal	0.8585	0.8937
HPO GRU	Normal	0.8524	0.8893
HPO TCN	Normal	0.8470	0.8843
CNN	Normal	0.8383	0.8774
HPO TCN	Augmented	0.8647	0.8982
HPO CNN	Augmented	0.8642	0.8977
HPO Mamba	Augmented	0.850	0.8870
Ensemble CNN+TCN	Augmented	0.8587	0.8932
RNN	Pure	0.8808	0.9063
DS RNN_B	Noise	0.7981	0.8458
HPO LSTM	Noise	0.7908	0.8417

Table V compares against classical ML [2]. DL improves combined data by 13.6% and noise by 3.6%, but trails on pure by 4.4%. DL’s advantage emerges where both training data and interference complexity are sufficient.

C. Architecture Performance and HPO

Table VI lists the top models per variant. Among single architectures, CNN (MCC 0.838) and TCN (0.836) lead. Mamba reaches 0.800, outperforming the Transformer at 0.713. DeepStackEnsemble achieves the best combined MCC of 0.859.

Table VII shows HPO impact. Recurrent architectures benefit most (GRU +20.0%, LSTM +23.1%, BiLSTM +19.9%), revealing severe under-parameterisation at default settings. CNN and TCN gain less. Complete HPO configurations are provided as supplementary material [19].

D. Augmentation Ablation

Table VIII shows augmentation impact. CNNs and TCNs gain from synthetic diversity (0.011 to 0.029 MCC); recurrences may degrade (0.001 to 0.006). Synthetic noise creates local fluctuations that recurrences interpret as temporal dependencies; CNNs smooth them through weight sharing.

Table VII
HPO GAINS ON NORMAL VARIANT

Model	Default MCC	HPO MCC	Gain
GRU	0.7101	0.8524	+20.0%
LSTM	0.6855	0.8435	+23.1%
BiLSTM	0.6838	0.8195	+19.9%
RNN	0.7552	0.8319	+10.2%
Mamba	0.8002	0.8418	+5.2%
TCN	0.8356	0.8470	+1.4%
CNN	0.8383	0.8357	−0.3%

Table VIII
AUGMENTATION IMPACT (NORMAL VS. AUGMENTED MCC)

Model	Normal	Augmented	Change
CNN	0.8383	0.8490	+1.0128%
TCN	0.8356	0.8344	−0.9986%
GRU	0.7101	0.7039	−0.9913%
LSTM	0.6855	0.6804	−0.9926%
Mamba	0.8002	0.8073	+1.0089%
HPO TCN	0.8470	0.8647	+1.0209%
HPO CNN	0.8357	0.8642	+1.0341%

Table IX
ENSEMBLE STRATEGY COMPARISON (NORMAL VARIANT)

Strategy	MCC	Description
CNN (best single)	0.8383	No HPO or ensemble
HPO GRU (best HPO)	0.8524	50 Optuna trials
Stacking (LR)	0.8488	Logistic regression on 9 logits
DeepStackEnsemble	0.8585	Two-stage learned weighting
Ensemble All-9 (vote)	0.8074	Uniform soft voting

E. Ensemble Strategies

Table IX compares ensemble strategies. CNN (0.838) through HPO GRU (0.852) and stacking (0.849) to DeepStack (0.859) shows incremental gains. Uniform voting degrades to 0.807 because weak classifiers contribute noisy probabilities. DeepStack avoids this through per-class learned weighting. The stacking meta-learner receives logits from the same 9 architectures (config A, Table II).

F. Per-Class Performance

Table X presents per-class F1. AB leads (0.872–0.941) from its distinctive descending trajectory. AA is hardest (0.825–0.890) due to signal saturation near the AP. Noise degrades all classes, with BB dropping most (0.933 to 0.832).

G. Noise and Interference Analysis

Figure 1 shows RSSI shift per (Route $\times$ Interferer). AB interferer produces largest shifts (up to +4.6 dBm on AA); AB is most affected (−2.4 dBm from AA). Figure 2 shows the RSSI trajectories per class under interference.

Per-timestep perturbation peaks at timesteps 4 to 6. MetaFusion_GRU reaches MCC 0.783 on noise, compared to 0.758 for standard GRU.

H. Feature Importance

Permutation importance measures the MCC drop when a given feature’s values are randomly shuffled, breaking its

Table X
PER-CLASS F1 BY VARIANT (BEST MODEL)

Variant	AA	AB	BA	BB
Pure	0.889	0.941	0.857	0.933
Noise	0.825	0.872	0.857	0.832
Normal	0.866	0.925	0.895	0.873
Augmented	0.890	0.929	0.902	0.875

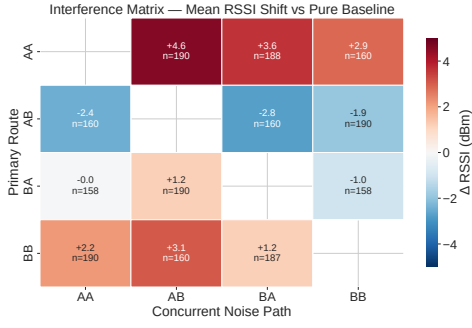


Figure 1. Interference heatmap: mean RSSI shift per (Route $\times$ Interferer) pair.

relationship with the target while preserving the marginal distribution. Figure 3 shows this metric across the four engineered channels and ten timesteps, averaged over CNN, TCN, and GRU. Window deviation dominates early timesteps ($t = 1$ to 3). Velocity peaks at $t = 4$; raw RSSI at $t = 6$; acceleration at $t = 3$ to 5.

I. Discussion

Classical and deep methods complement each other. KNN leads on clean single-device data (0.922 vs. 0.881) because local neighbourhood structure directly captures clean RSSI trajectories without training. DeepStack leads under multi-device interference (0.859 vs. 0.770) because learned feature hierarchies disentangle overlapping signals. This complementarity suggests density-based routing: KNN for sparse single-device conditions, DeepStack for dense multi-device scenarios.

The feature importance analysis (Figure 3) extends prior findings [2], where classical ML on raw RSSI identified early timesteps as most important (MCC 0.756). DL additionally captures mid-trajectory dynamics through the velocity channel ($t = 4$, $\Delta\text{MCC } 18.4 \times 10^{-3}$), a signal classical models could not exploit. The window deviation channel explains BB’s noise resilience: trajectory shape survives interference-level shifts that distort absolute RSSI. The trajectory analysis (Figure 2) confirms this mechanism: AA flattens AB’s characteristic 20 dBm descent into a constant profile, erasing the primary discriminative feature. BB’s flat pattern shifts in magnitude but retains its shape.

The HPO gain gap between recurrent architectures (20–23%) and CNN/TCN ($\pm 1\%$) reflects a fundamental difference in inductive bias. CNN kernels operate on fixed-size windows regardless of sequence length. TCN dilated convolutions naturally match 10-step sequences through exponentially growing receptive fields. Recurrent hidden states must encode all temporal context into a fixed-size vector. At default dimension 64, this creates an information bottleneck. HPO discovers larger hidden dimensions that remove the bottleneck.

Uniform voting across all nine architectures (MCC 0.807) underperforms a single CNN (0.838). Weak classifiers contribute noisy probability estimates that shift the decision boundary. DeepStack (0.859) avoids this through per-class

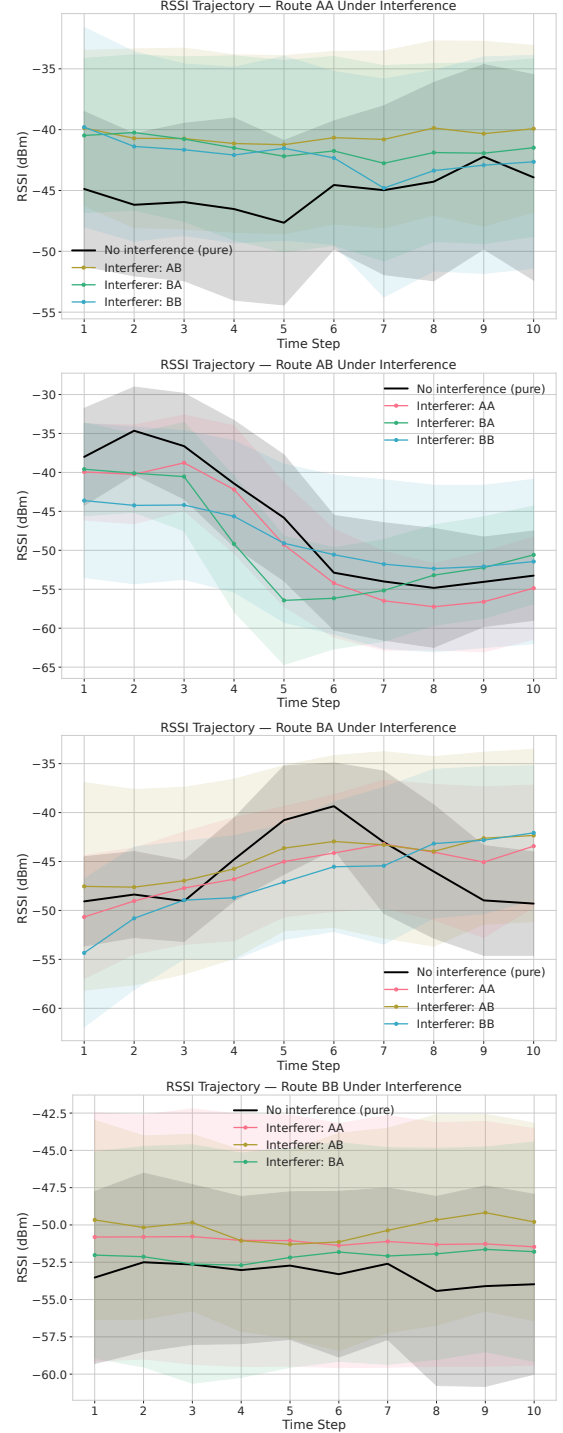


Figure 2. Trajectories under interference for all four classes.

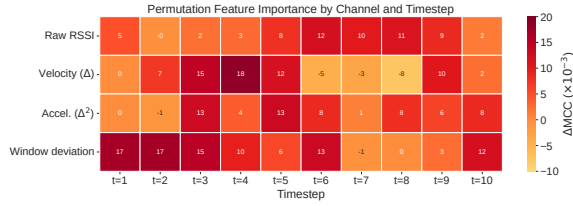


Figure 3. Permutation feature importance by channel and timestep (CNN, TCN, GRU average).

Level2Nets that learn class-specific base model importance. Two findings point toward future work. Mamba’s $O(L)$ complexity enables scaling to longer observation windows of 20–30 seconds, where self-attention’s quadratic cost becomes prohibitive. MetaFusion_GRU (MCC 0.783 on noise vs. 0.758 baseline) shows that context-aware architectures can partially disentangle interference.

Limitations include the controlled laboratory environment, fixed 10-second window, MAC address randomisation, and untested device heterogeneity across smartphone models. For deployment, CNN (28K parameters) enables real-time on-device inference at the access point. DeepStackEnsemble (1.2M parameters) targets server-side batch inference where latency permits evaluating all 18 base models.

V. CONCLUSION

This paper evaluated deep learning architectures for RSSI-based passenger movement classification in public transport. The two-stage DeepStackEnsemble achieved MCC 0.859 and accuracy 89.4% on combined data, a 13.6% relative improvement over classical baselines. State-space models reached up to MCC 0.850 with $O(L)$ complexity. Cross-route interference degraded performance by 9.4%, with AB identified as both the dominant interferer and the most affected route. Feature importance analysis showed that DL captures mid-trajectory motion dynamics through derivative channels that classical ML could not exploit, contributing directly to the improvement.

Future work should explore attention mechanisms that down-weight interfered timesteps, multi-AP fusion for spatial diversity, and integration with origin-destination inference for end-to-end flow modeling. Context-aware architectures that jointly model signal and environmental metadata offer a promising direction for real-world deployment.

VI. GENERATIVE AI DECLARATION

During the preparation of this manuscript, the authors used DeepSeek and Gemini as a writing assistant for language improvement, grammar correction, and formatting. All technical content, experimental design, model architectures, results, analysis, and conclusions represent the authors’ original intellectual contribution. The authors take full responsibility for the accuracy and integrity of the content.

ACKNOWLEDGMENT

This work is supported by the European Regional Development Fund (FEDER), through the Regional Operational

Programme of Centre (CENTRO 2030) of the Portugal 2030 framework [Project inMotion with Nr. 21359 (CENTRO2030-FEDER-02225900)].

REFERENCES

- [1] European Environment Agency, “Greenhouse gas emissions from transport in europe,” <https://www.eea.europa.eu/en/analysis/indicators/greenhouse-gas-emissions-from-transport>, 2025, published 06 Nov 2025.
- [2] A. Ribeiro, G. Matos, J. Corona, M. Antunes, and D. Gomes, “Rssi-based passenger movement classification for non-intrusive public transport monitoring,” in *2026 Joint European Conference on Networks and Communications & 6G Summit (EuCNC/6G Summit)*, 2026, submitted.
- [3] C. Pronello and X. R. Garzón Ruiz, “Evaluating the performance of video-based automated passenger counting systems in real-world conditions: A comparative study,” *Sensors*, vol. 23, no. 18, p. 7719, 2023.
- [4] K. Ono, T. Uchibayashi, C. Sueyoshi, K. Inenaga, and Y. Yasutake, “Real-time passenger detection and counting using edge ai camera,” in *2025 IEEE Cyber Science and Technology Congress (CyberSciTech)*, 2025, pp. 606–609.
- [5] C. Wiboonsiriruk, E. Phaisangittisagul, C. Srisurangkul, and I. Kumazawa, “Efficient passenger counting in public transport based on machine learning,” in *TENCON 2023 - 2023 IEEE Region 10 Conference (TENCON)*, 2023, pp. 214–219.
- [6] M. Nitti, F. Pinna, L. Pintor, V. Pilloni, and B. Barabino, “iabacus: A wi-fi-based automatic bus passenger counting system,” *Energies*, vol. 13, no. 6, p. 1446, 2020.
- [7] T. A. Myrvoll, J. E. Håkegård, T. Matsui, and F. Septier, “Counting public transport passenger using wifi signatures of mobile devices,” in *2017 IEEE 20th International Conference on Intelligent Transportation Systems (ITSC)*, 2017, pp. 1–6.
- [8] L. Fabre, C. Bayart, Y. Kone, O. Manout, and P. Bonnel, “A machine learning approach to estimate public transport ridership using wi-fi data,” *IEEE Transactions on Intelligent Transportation Systems*, vol. 26, no. 1, pp. 906–915, 2025.
- [9] A. Simončič, M. Mohorčič, M. Mohorčič, and A. Hrovat, “Non-intrusive privacy-preserving approach for presence monitoring based on wifi probe requests,” *Sensors*, vol. 23, no. 5, p. 2588, 2023.
- [10] H. Wang and I. W.-H. Ho, “Csi-based passenger counting on public transport vehicles with multiple transceivers,” in *2024 IEEE Wireless Communications and Networking Conference (WCNC)*, 2024, pp. 1–6.
- [11] N. Singh, S. Choe, and R. Punmiya, “Machine learning based indoor localization using wi-fi rssi fingerprints: An overview,” *IEEE Access*, vol. 9, pp. 127 150–127 174, 2021.
- [12] A. Gu and T. Dao, “Mamba: Linear-time sequence modeling with selective state spaces,” *arXiv preprint arXiv:2312.00752*, 2023.
- [13] A. Lahoti, K. Y. Li, B. Chen, C. Wang, A. Bick, J. Z. Kolter, T. Dao, and A. Gu, “Mamba-3: Improved sequence modeling using state space principles,” *arXiv preprint arXiv:2603.15569*, 2026.
- [14] H. Wu, J. Xu, J. Wang, and M. Long, “Autoformer: Decomposition transformers with auto-correlation for long-term series forecasting,” *Advances in Neural Information Processing Systems*, vol. 34, pp. 22 419–22 430, 2021.
- [15] N. Shazeer, A. Mirhoseini, K. Maziarz, A. Davis, Q. Le, G. Hinton, and J. Dean, “Outrageously large neural networks: The sparsely-gated mixture-of-experts layer,” *arXiv preprint arXiv:1701.06538*, 2017.
- [16] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, “Optuna: A next-generation hyperparameter optimization framework,” in *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, 2019, pp. 2623–2631.
- [17] S. Bai, J. Z. Kolter, and V. Koltun, “An empirical evaluation of generic convolutional and recurrent networks for sequence modeling,” *arXiv preprint arXiv:1803.01271*, 2018.
- [18] D. Chicco and G. Jurman, “The advantages of the matthews correlation coefficient (mcc) over f1 score and accuracy in binary classification evaluation,” *BMC Genomics*, vol. 21, no. 1, p. 6, 2020.
- [19] A. Ribeiro, G. Matos, J. Corona, and M. Antunes, “Hyperparameter optimization supplementary materials,” <https://github.com/ATNoG/inMotion/tree/main/docs/icaif>, 2026, accessed: 2026-06-10.